embarca tech | Residência Tecnológica em Sistemas Embarcados

SBC Labrador

RESIDÊNCIA – Aula Síncrona (30/09/2025)

Prof. Dr Ricardo Menezes Prates

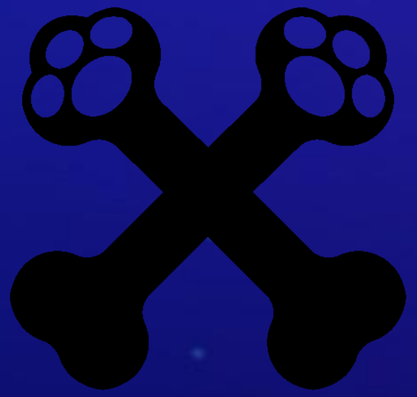
Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

- *Introdução*
  - Planejamento da aula

- Computador Labrador – Single Board Computer (SBC)
  - *Programação em C*
  - *Manipulação de GPIOs*
  - *Configuração do periférico PWM*

- Conclusões

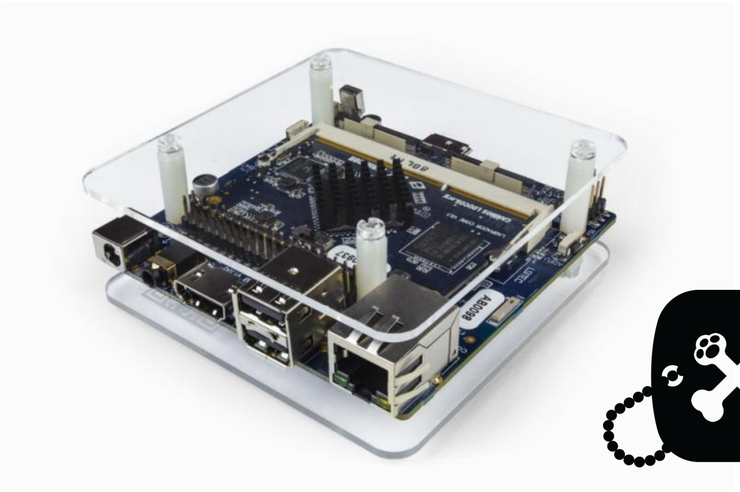
Introdução



Objetivo

- **Objetivo**: Programação de periféricos da SBC Labrador através da linguagem de programação C e da IDE Code::Clocks.

- **Atividade**: Manipulação das portas GPIO e Implementação de código para configuração do módulo PWM.



Placa Labrador

Planejamento da Aula

- Pré-requisitos para esta aula:

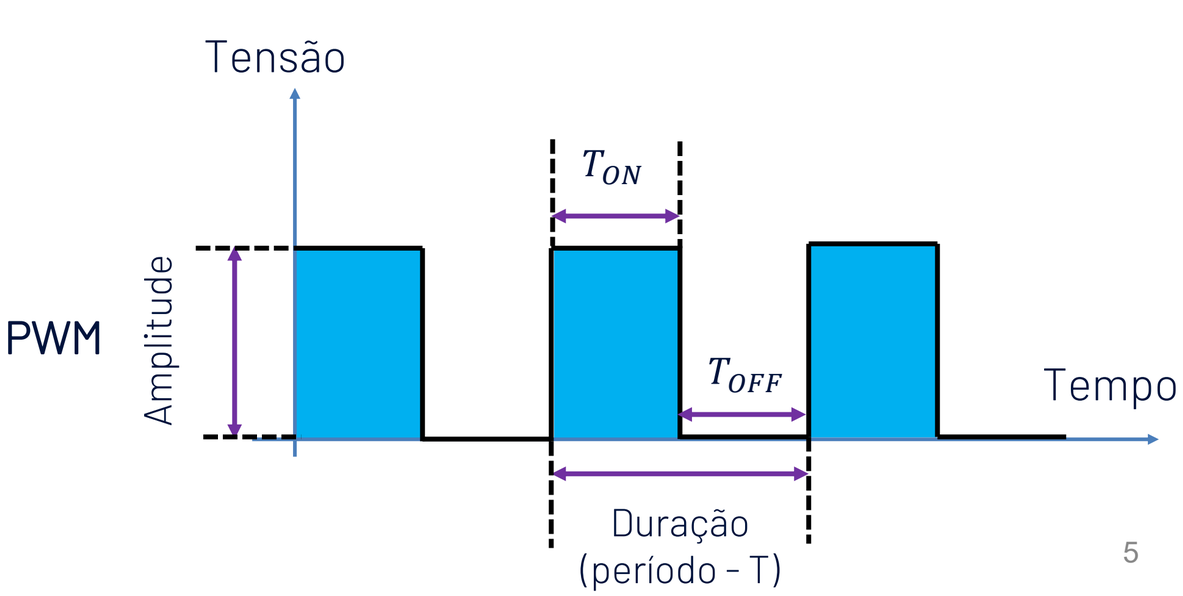
✓ Linguagem C;

✓ *Interface de comunicação;*

✓ *Linux básico;*

✓ Portas de Entrada e Saída (GPIO);

✓ PWM.



Planejamento da Aula

- Potenciais perguntas a serem trabalhadas nesta aula.

✓ É possível manipular os pinos GPIO da SBC Labrador utilizando a linguagem C?

✓ Como trabalho com a Modulação em Largura Pulsos (PWM) utilizando o periférico presente placa labrador?

✓ Existe alguma IDE indicada para computadores com restrição de hardware?

Code::Blocks

Labrador

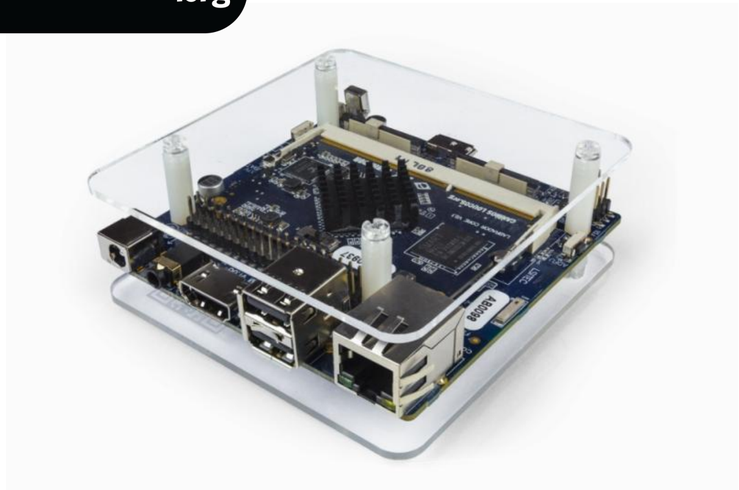
Labrador – Revisão

- Computador Labrador:

✓ É um computador de placa única (*SBC – Single Board Computer*) criado pelo projeto brasileiro Caninos Loucos.

✓ É uma SBC brasileira, com hardware e software aberto, voltada para aplicações em IoT.

✓ Possui arquitetura modular (Módulo Central + Placa Base), permitindo flexibilidade para educação, pesquisa, IoT e aplicações industriais.



https://caninosloucos.org/pt/

Labrador - Revisão

- Computadores - SBC:

✓ Existem várias placas similares no mercado que têm propósito parecido. Oferecer um computador:

1. Compacto,
2. Tecnologia de baixo custo e baixo consumo,
3. Direcionado para educação, IoT, robótica e indústria.

Raspberry Pi 4

Jetson Nano

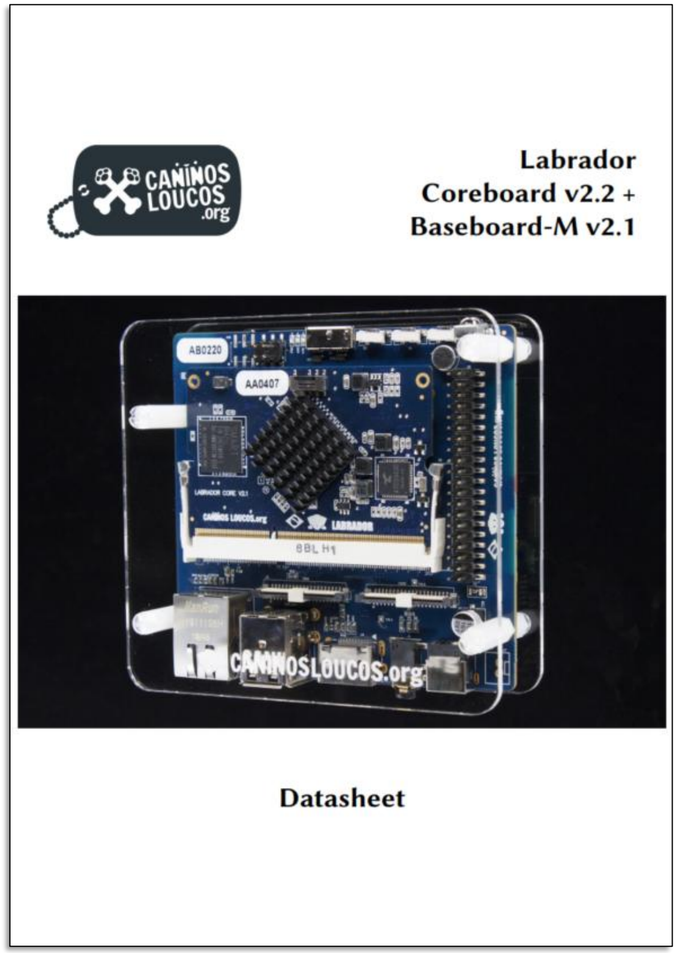
Orange Pi

Labrador - Revisão

- Computador Labrador – arquitetura modular (*System on Module – SoM*) :

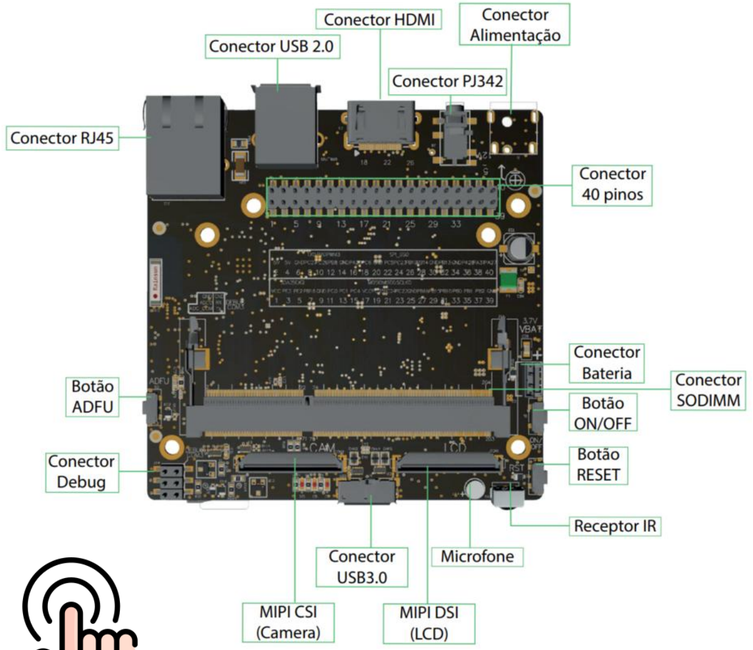
✓ Módulo central (*Core Board*): Contém o processador (ARM Cortex-A), memória RAM e armazenamento (eMMC). Pode ser atualizado/trocado sem mudar toda a placa base.

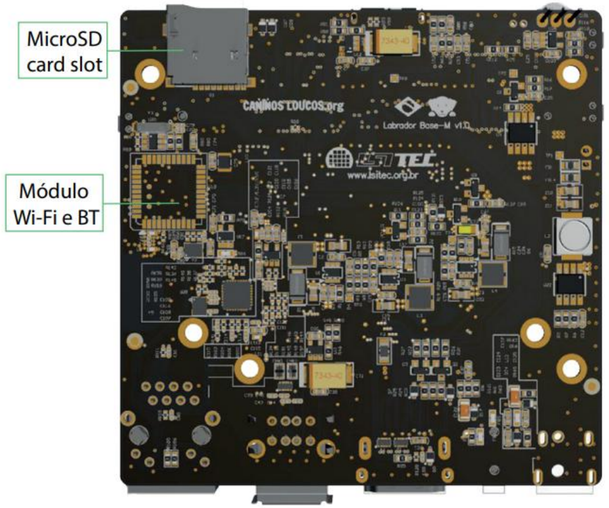
✓ Placa base (Base Board): Fornece as conexões físicas e periféricos: USB, HDMI, GPIOs, conectividade sem fio, etc.



Labrador – revisão

- Computador Labrador – Placa base:





BASE BOARD M V.1.0

Armazenamento: MicroSD Card Slot SD/SDHC/SDXC - até: 32GB
Ethernet: 10/100Mbps (RJ45)
Wireless: Wi-Fi 802.11 b/g/n 2.4GHz
Bluetooth 4.0
1 x receptor infravermelho (38kHz)
USB: 2 x USB2.0 HOST (tipo A)
1 x USB3.0 OTG (micro-B)
Display: 1 x HDMI 1.4 (tipo A), até 1920x1080@60Hz
1 x LVDS-DSI para LCDs, até 1920x1080@60Hz
1 x CVBS PAL/NTSC (PJ342 3,5mm)
Áudio: Saída no HDMI
Saída analógica estéreo (PJ342 3,5mm)
I2S entrada/saída
Microfone embutido
Câmera: 1 x MIPI-CSI
1 x interface paralela de 8-bits
LED: 1 x ligar/desligar (vermelho)
1 x programável (verde)
1 x programável (azul)
Botões: 1x ligar/desligar
1x reiniciar
1x ADFU
Alimentação: 5~12V@2A
(diâmetro interno 2,1mm, externo de 5,5mm, centro positivo)
Expansão: Conector SODIMM de 204 pinos (fêmea), 40-pin header: 28 GPIOs
(compatível com Raspberry Pi / suporta UART, I2C, SPI, PWM e I2S)
ADC input
Debug: UART
Dimensões: 88mm x 81,3mm
Peso: 52,6g

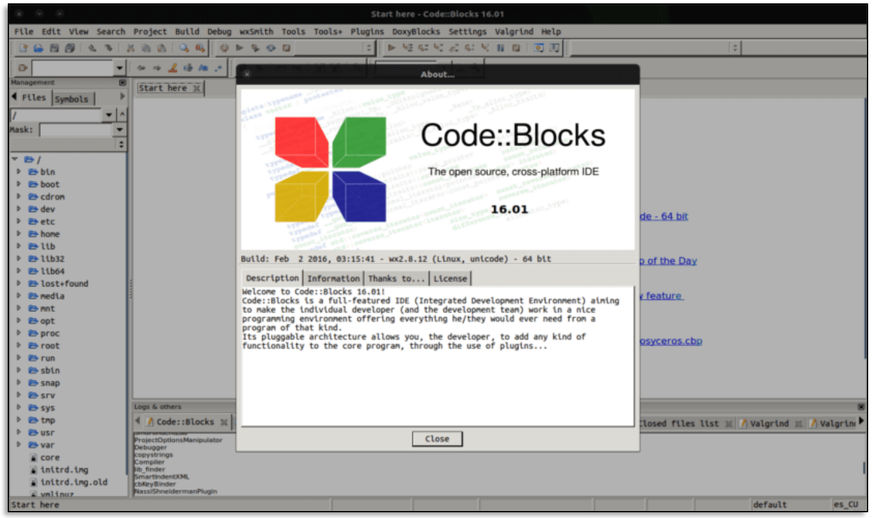
Labrador – Code::Blocks

- Code::Blocks:

✓ Ambiente de desenvolvimento integrado (IDE), gratuito e de código aberto, que permite programar em linguagens como C, C++ e Fortran em diferentes sistemas operacionais (Windows, Linux, macOS).

✓ Inclui recursos, a exemplo de realce de sintaxe;

✓ **Suporte a Múltiplos Compiladores**: Conecta-se a diversos compiladores como GCC, Clang e Visual C++, adaptando-se a diferentes necessidades.



Labrador – Biblioteca gpiod

- ✓ A libgpiod (*General-Purpose Input/Output Device library*) é uma biblioteca em C que fornece acesso às GPIOs em Linux.
- ✓ Substitui a interface baseada em /sys/class/gpio.
- ✓ Ela usa a *GPIO character device interface* (introduzida no Linux kernel 4.8) para manipular os pinos de forma mais eficiente e estruturada.

Conector de 40 pinos - GPIO Labrador 32-bits

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	**GPIOC27**/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	**GPIOC26**/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	**GPIOB8**/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	**GPIOA25**/SIRQ1
3.3V	N/C	17	18	149	**GPIOC6**/DSI_DP0/UART2_RX/SPIO_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	**GPIOC5**/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	**GPIOC23**/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	**GPIOB19**/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	**GPIOB14**/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	**GPIOB13**/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	**GPIOA28**/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	**GPIOA31**/I2S_D1
GND	N/C	39	40	3	**GPIOA27**/I2S_D0

Manipulação de GPIOs

- Manipulação de GPIO:

✓ A Labrador possui 5 grupos de GPIO (A, B, C, D e E). Cada um desses possui 32 pinos agrupados, portanto para indicar o valor absoluto do pino temos um offset dado pelo grupo mais o valor do pino, este offset seguem a lógica a seguir:

- A: 0
- B: 32
- C: 64
- D: 96
- E: 128

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	GPIOC27/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	GPIOC26/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	GPIOB8/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	GPIOA25/SIRQ1
3.3V	N/C	17	18	149	GPIOC6/DSI_DP0/UART2_RX/SPIO_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	GPIOC5/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	GPIOC23/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	GPIOB19/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	GPIOB14/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	GPIOB13/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	GPIOA28/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	GPIOA31/I2S_D1
GND	N/C	39	40	3	GPIOA27/I2S_D0

Manipulação de GPIOs

- Manipulação de GPIO:

GPIO 128+3=131

- Digite as seguinte linhas no terminal:

```
$ cd /sys/class/gpio #Muda o diretório atual - permite controlar as GPIOs.

$ echo 131 | sudo tee export #Exportar GPIOE3

$ cd gpio131 #Acessar gpio131

$ echo out | sudo tee direction #Definir como output

$ echo 1 | sudo tee value #Escrever no pino configurado - ON

$ echo 0 | sudo tee value #Escrever no pino configurado - OFF

$ cat value #Ler o valor do pino
```

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	GPIOC27/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	GPIOC26/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	GPIOB8/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	GPIOA25/SIRQ1
3.3V	N/C	17	18	149	GPIOC6/DSI_DP0/UART2_RX/SPI0_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	GPIOC5/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	GPIOC23/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	GPIOB19/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	GPIOB14/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	GPIOB13/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	GPIOA28/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	GPIOA31/I2S_D1
GND	N/C	39	40	3	GPIOA27/I2S_D0

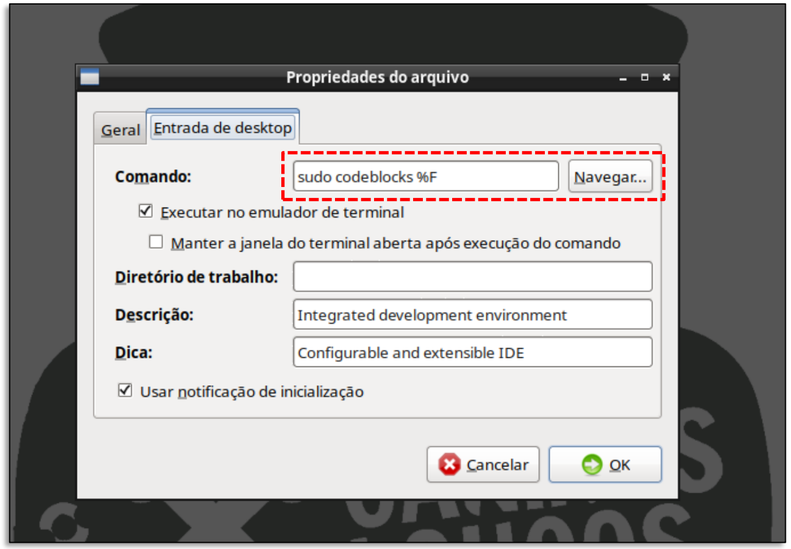
Manipulação de GPIOs

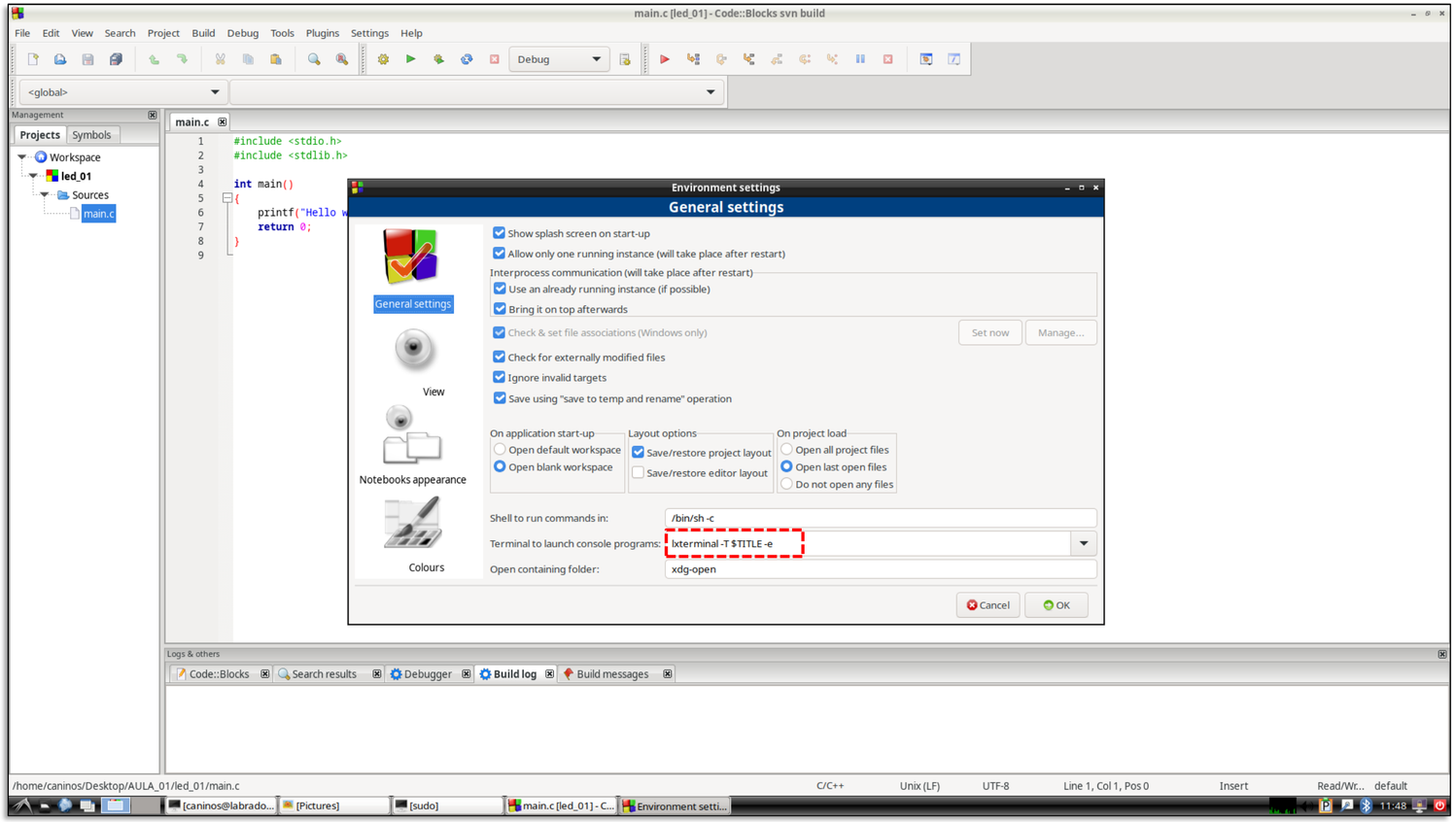
- Linguagem C:

```c
#include <gpiod.h> // Biblioteca para manipular GPIOs em sistemas linux
#include <stdio.h> // Biblioteca para interface padrão em C
#include <unistd.h> // Biblioteca dá acesso a funções especiais no sitema operacional - ex: usleep.


//Função principal
int main() {

    // Estrutura de dados para representação da GPIO
    struct gpiod_chip *chip;
    struct gpiod_line *line;
    int ret;

    // Abre a conexão com um chip GPIO com um número específico
    chip = gpiod_chip_open_by_number(4);
    if (!chip) {
        perror("Erro ao abrir chip GPIO\n");
        return 1;
    }

    // Obtém uma linha específica do chip
    int gpio = 2;
    line = gpiod_chip_get_line(chip, gpio);
    if (!line) {
        perror("Erro ao acessar linha GPIO\n");
        gpiod_chip_close(chip);
        return 1;
    }

    // Configurar a GPIO como saída e seta valor inicial zero
    ret = gpiod_line_request_output(line, "gpio_toggle", 0);
    if (ret < 0) {
        perror("Erro ao requisitar linha como saida");
        gpiod_chip_close(chip);
        return 1;
    }

    // Piscar o LED conectado a GPIOE3
    for (int i = 0; i < 10; i++) {
        gpiod_line_set_value(line, 1);
        usleep(500000);
        gpiod_line_set_value(line, 0);
        usleep(500000);
        printf("Pisca LED\n");
    }

    // Liberar uma linha GPIO que foi previamente requisitada
    gpiod_line_release(line);
    // Fechar conexão com o CHIP GPIO
    gpiod_chip_close(chip);
    return 0;
}
```

Manipulação de GPIOs

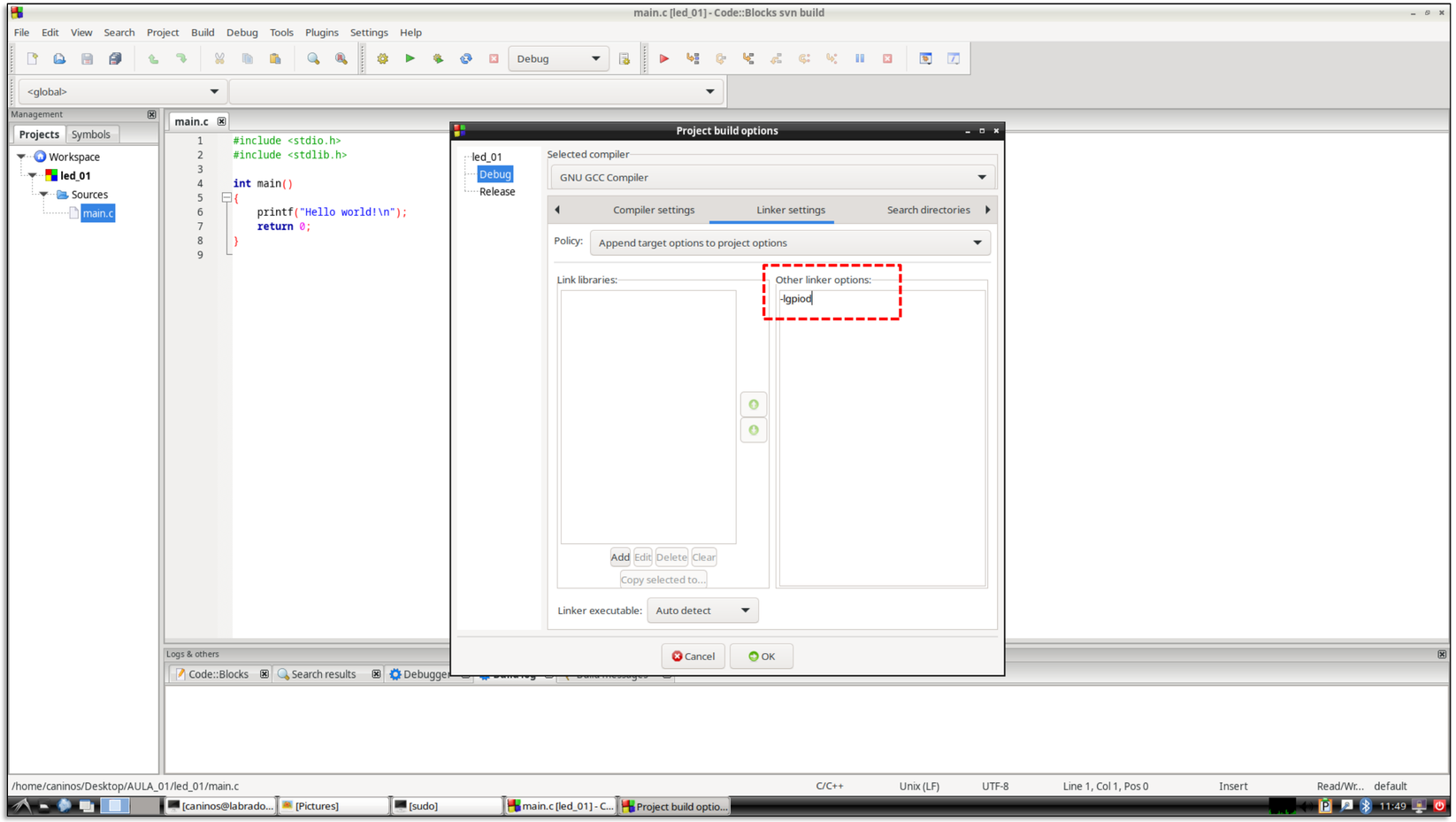
- Dicas – Code::Blocks:





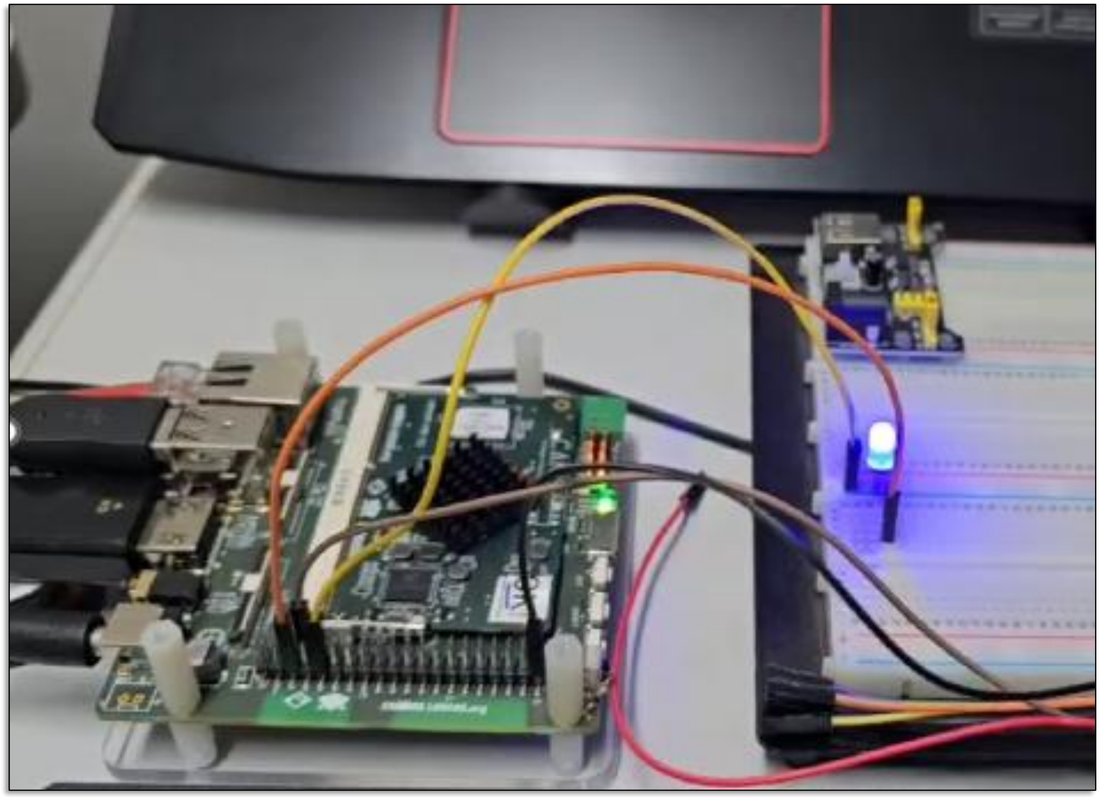
Manipulação de GPIOs

- Dicas – Code::Blocks:



Manipulação de GPIOs

- Experimento:



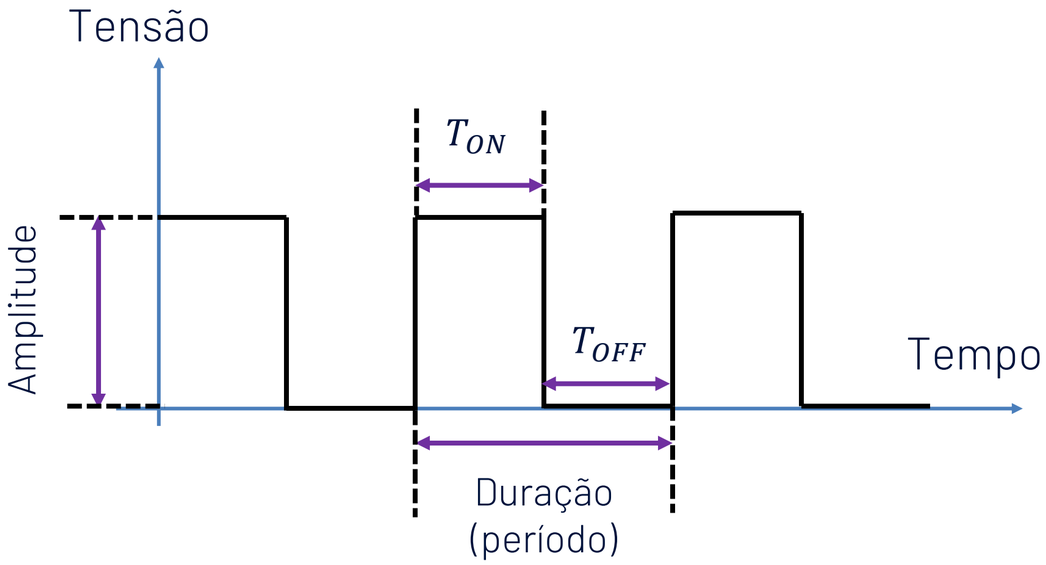
PWM - Revisão

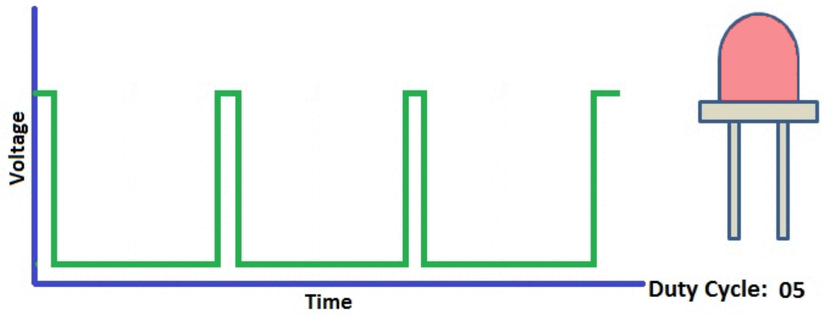
- Características:

✓ A **modulação por Largura de Pulsos** (em inglês, *Pulse Width Modulation* - PWM) é uma técnica que envolve a variação da largura dos pulsos em um ciclo do sinal elétrico.

✓ O **sinal PWM** é composto por uma série de pulsos retangulares com largura variável.

✓ É realizado o ajuste do tempo que o sinal fica em **nível alto** em cada ciclo para controlar a tensão média e a potência fornecida.





Fonte: www.citisystems.com.br/pwm/

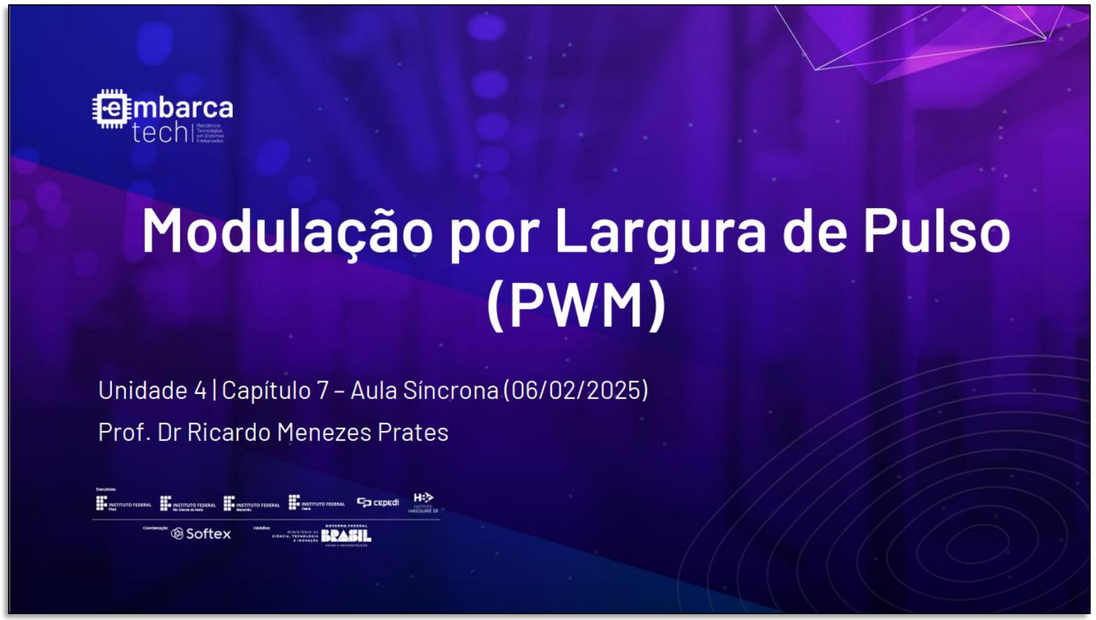
PWM - Revisão

- Características:

✓ A **modulação por Largura de Pulsos** (em inglês, *Pulse Width Modulation* - PWM) é uma técnica que envolve a variação da largura dos pulsos em um ciclo do sinal elétrico.

✓ O **sinal PWM** é composto por uma série de pulsos retangulares com largura variável.

✓ É realizado o ajuste do tempo que o sinal fica em **nível alto** em cada ciclo para controlar a tensão média e a potência fornecida.



PWM

- Comandos no terminal do Linux – GPIOB8:

1) Liste os dispositivos PWM disponíveis:

$ cd /sys/class/pwm

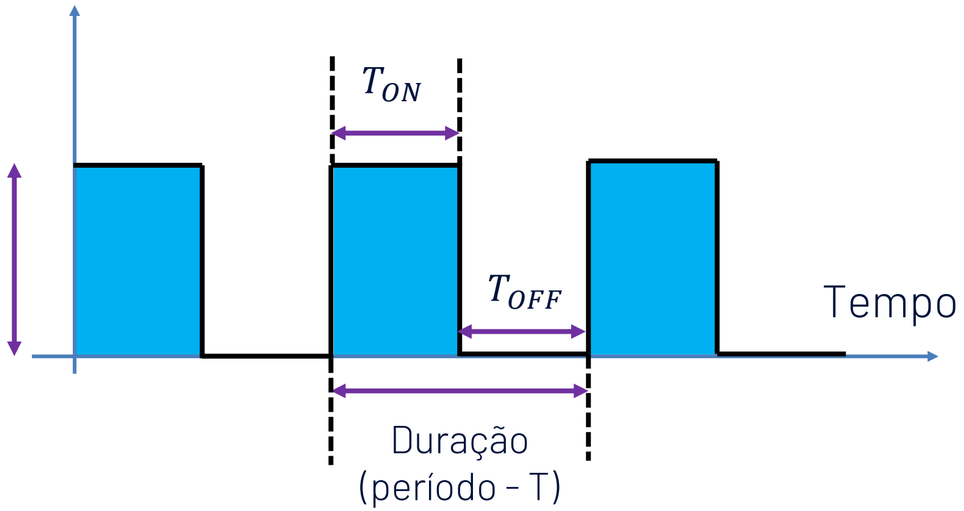
$ ls

pwmchip0

$ cd pwmchip0

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	**GPIOC27**/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	**GPIOC26**/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	**GPIOB8**/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	**GPIOA25**/SIRQ1
3.3V	N/C	17	18	149	**GPIOC6**/DSI_DP0/UART2_RX/SPIO_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	**GPIOC5**/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	**GPIOC23**/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	**GPIOB19**/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	**GPIOB14**/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	**GPIOB13**/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	**GPIOA28**/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	**GPIOA31**/I2S_D1
GND	N/C	39	40	3	**GPIOA27**/I2S_D0

PWM



- Comandos no terminal do linux:

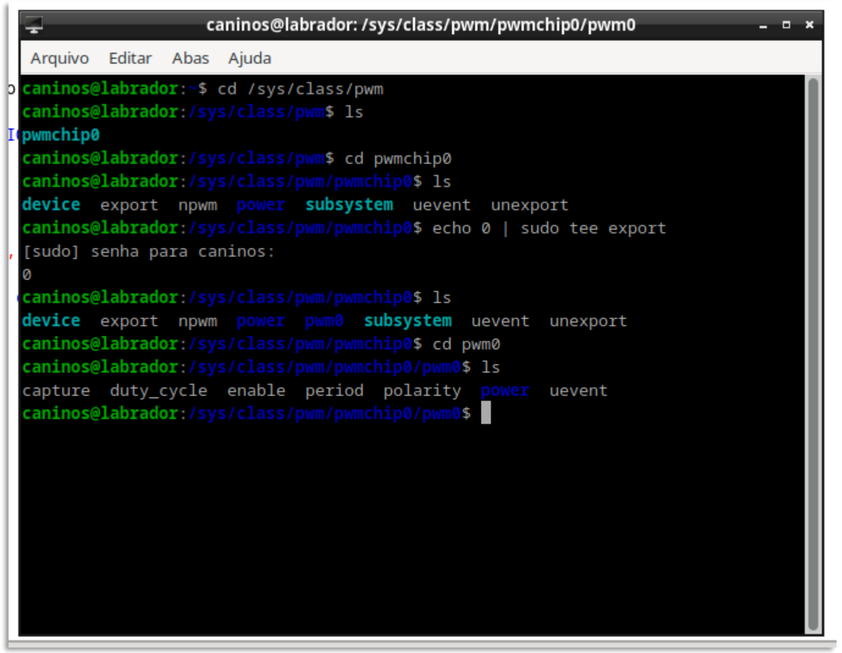
2) Exporte o canal que deseja usar (ex: canal 0):

$ echo 0 | sudo tee export

3) Configura o período (em nanossegundos, por exemplo 20 ms = 20000000 ns):

$ cd pwm0

$ echo 20000000 | sudo tee period



PWM

- Comandos no terminal do linux:

4) Configure o duty cycle (ex: 50% → 10 ms = 10000000 ns):

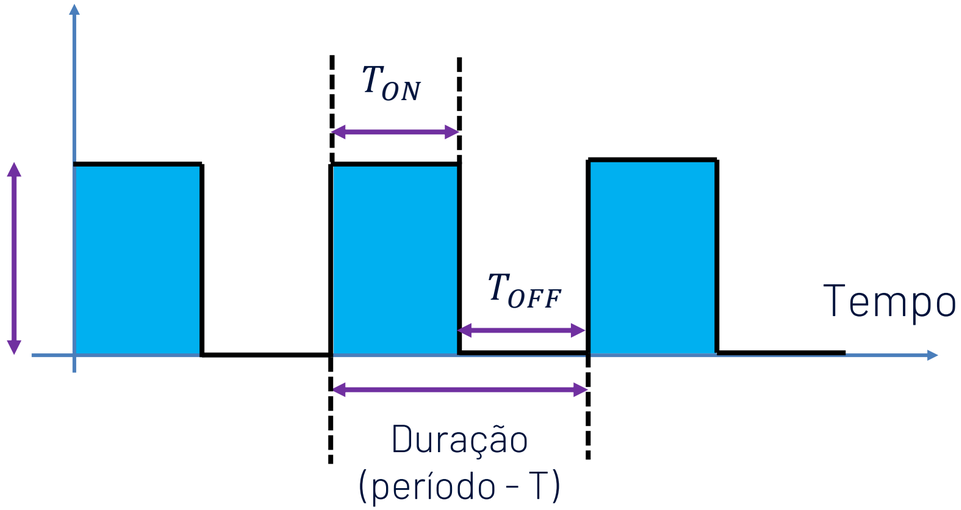
$ echo 10000000 | sudo tee duty_cycle

5) Ativar o PWM:

$ echo 1 | sudo tee enable

6) Desativar o PWM:

echo 0 | sudo tee enable



$$D_C = \left(\frac{T_{ON}}{T_{ON} + T_{OFF}}\right) \times 100$$

PWM

- Programação em C:

Trecho do código Parte 1/2

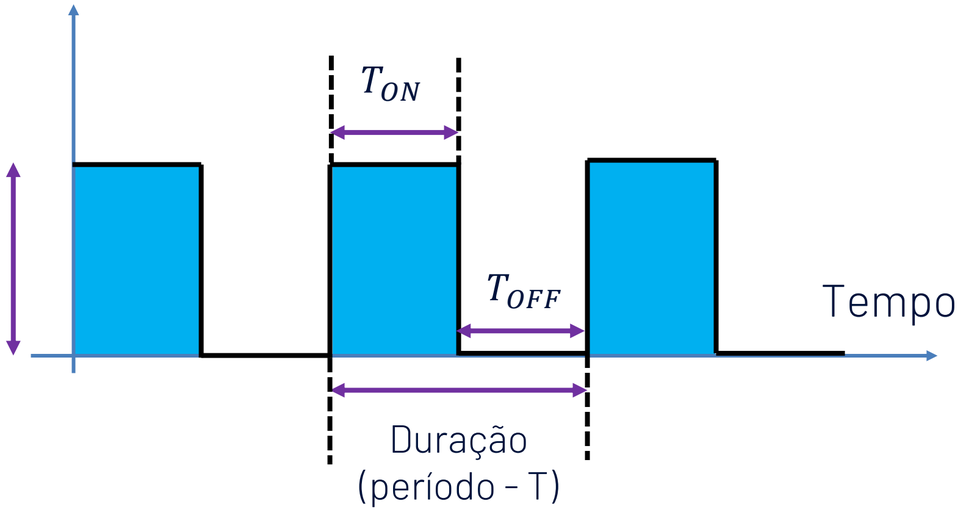
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>


// Define as macros para os diretórios pwm_chip e pwm0
#define PWM_CHIP "/sys/class/pwm/pwmchip0"
#define PWM0 PWM_CHIP "/pwm0"


// Função que recebe um caminho de arquivo (path) e um valor em string - abre o arquivo no modo escrita.
// Esta função é necessária devido ao linux controlar o pwm via arquivos especiais em sysfs - sistemas de arquivos virtuais do
linux.
void writeToFile(const char *path, const char *value){


    FILE *fp = fopen(path, "w");
    if(fp == NULL){
        perror("Erro ao abrir arquivo");
        exit(1);
    }
    fprintf(fp, "%s", value);
    fclose(fp);
}
```

PWM

- Programação em C:

Trecho do código Parte 2/2

```c
// Função principal
int main()
{
    //Exporat a PWM0
    writeToFile(PWM_CHIP "/export", "0");


    //Configurar o período para 20 ms
    writeToFile(PWM0 "/period", "20000000");


    //Configurar duty cycle para 50%
    writeToFile(PWM0 "/duty_cycle", "10000000");


    //Ativar PWM
    printf("Ativando PWM!\n");
    writeToFile(PWM0 "/enable", "1");


    //intervalo de 30 segundos
    sleep(30);

    //Desativar PWM
    printf("Destivando PWM!\n");
    writeToFile(PWM0 "/enable", "0");
    sleep(5);


    return 0;
}
```



Conclusões

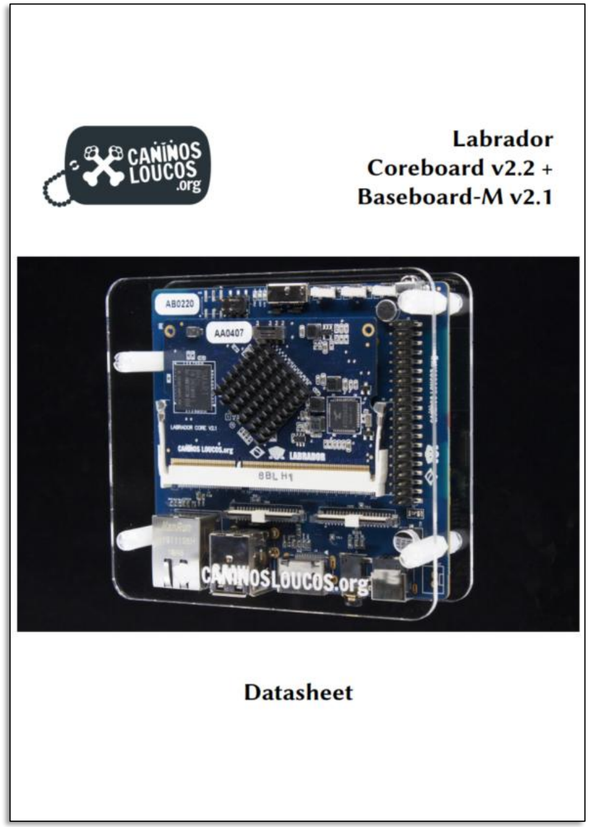
Conclusões

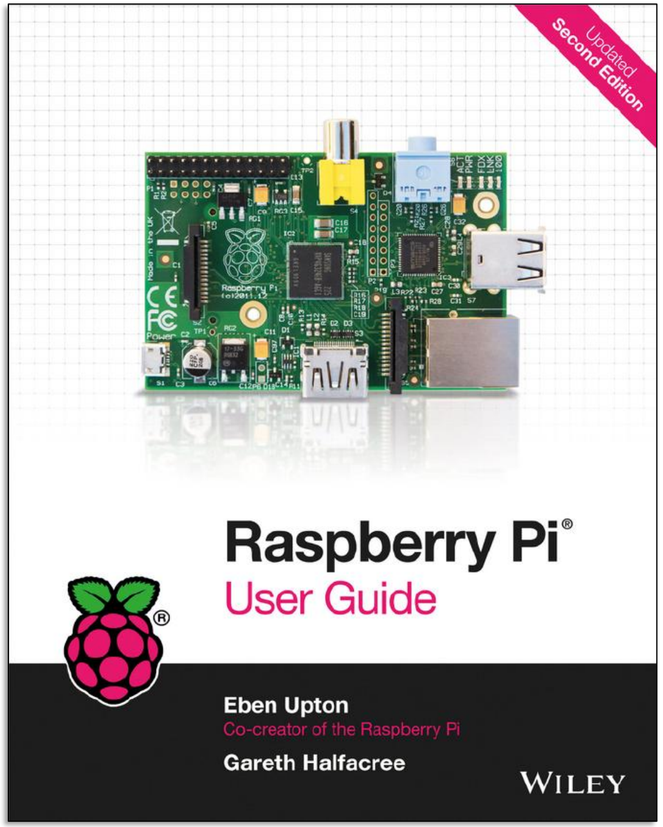
- A SBC labrador é uma ferramenta versátil e conta com periféricos que viabilizam o desenvolvimento de diversos sistemas embarcados.

- A linguagem C pode ser utilizada para realizar o controle das GPIOs e do módulo PWM, que são periféricos disponíveis na arquitetura modular da placa labrador.

- É possível utilizar Ambientes de Desenvolvimento Integrado (IDEs), a exemplo do Code::Blocks, para otimizar o desenvolvimento de aplicações.

Bibliografia Recomendada





embarca tech | Residência Tecnológica em Sistemas Embarcados

SBC Labrador

RESIDÊNCIA – Aula Síncrona (30/09/2025)

Prof. Dr Ricardo Menezes Prates

Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO